

Event-Driven GitHub Automation Bot

The Problem

Build and deploy a web app plus a bot that reacts to activity in a GitHub repository.

The flow you're building:

1. A user signs in to your app with their GitHub account and connects one of their repositories.
2. Your app receives webhooks from that repo when things happen on it (a new issue is opened, a pull request is opened, code is pushed).
3. When an event arrives, your app processes it and acts: it writes back to GitHub (adds a label or posts a comment) and sends a notification to a Slack channel.
4. A dashboard, visible only after the user logs in, shows a live log of everything the bot has done and lets the user configure simple rules (for example: "issues whose title contains bug → add the bug label and send a Slack alert").

It's a small product, but a real one. Getting it working end-to-end on a live URL requires you to set up and connect several external services correctly — that integration work is the heart of this exercise.

Core requirements (everyone must deliver these)

1. A deployed, publicly reachable web app (GitHub webhooks and OAuth callbacks cannot point at localhost).
 2. GitHub sign-in so a user can authenticate and connect a repository they own.
 3. A working webhook endpoint that receives at least two GitHub event types (e.g. issues and pull requests) and records them.
 4. The bot writes back to GitHub for at least one event type (add a label *or* post a comment via the GitHub API).
 5. The bot sends a Slack notification when a configured event occurs.
 6. A dashboard (behind login) showing a log of received events and the actions the bot took.
 7. A [README.md](#) that lets us run it locally and explains how you deployed it.
-

Stretch goals (raise your ceiling — aim here if you have the experience)

These are how stronger candidates distinguish themselves. You don't need all of them.

1. Configurable rules in the UI (match on keywords, author, labels, etc.) rather than hard-coded behavior.

2. An AI step: run the issue/PR text through an LLM to auto-summarize, suggest a label, or triage by priority, and show that in the notification and dashboard. (See "Using AI inside the app" below — keep it free.)
 3. Authenticate as a GitHub App (JWT signed with a private key, exchanged for installation tokens) instead of a plain OAuth app.
 4. Multi-repository support for a single user.
 5. Meaningful observability: structured logs, a visible history of failures and retries.
-

Quality bar — what "working" actually means

Treat this as something that will run unattended. That mindset is what we're grading. In particular:

1. It should not be foolable by forged or replayed requests hitting your endpoints.
 2. It should not do the same thing twice if the same event is delivered more than once (this happens — plan for it).
 3. It should not silently lose events if a downstream call or your own service is briefly unavailable.
 4. It must never expose secrets — not in the repo, not in client-side code, not in logs.
-

Constraints

- Everything must be free. No credit card, anywhere. If a service asks for card details, you've picked the wrong tier or the wrong service — switch. You should not spend any money to complete this.
- Use any tech stack and language you're comfortable with. Full-stack means front end, back end, data, and deployment — all yours.
- Deploy to a real public host.

Suggested free services (all have no-card free tiers)

- GitHub — OAuth/App, API, and webhooks are all free.
 - Database — [Neon](#) or [Supabase](#) (free Postgres, no card).
 - Notifications — Slack (free workspace + app; a Slack Incoming Webhook URL or a bot token both work, no card). Telegram via BotFather is a fine choice for the optional second channel.
 - Hosting — Render, Vercel, Cloudflare, or Netlify (free tiers, no card).
 - AI (only if you do the AI stretch goal) — Google Gemini (via Google AI Studio) or Groq. Both give an API key on a free tier with no credit card. Do not use a paid LLM API for this exercise.
-

Deliverables (your submission)

1. A GitHub repository with all your code and a clear commit history.

2. The deployed URL, working and reachable when we open it.
 3. A [README.md](#) covering: what the app does, how to run it locally, the environment variables it needs (provide a .env.example with no real secrets), and how/where you deployed it.
 4. A way for us to test it — brief instructions, and any throwaway test credentials or a demo repo we can point the webhook at if needed.
 5. Your AI context/instruction files, exactly as you used them — e.g. [CLAUDE.md](#), [AGENTS.md](#), .cursorrules, or equivalent. If you didn't use any, say so in AI_NOTES.md.
 6. AI_NOTES.md (about one page) — see below.
-

Using AI — and what to tell us about it

You should use AI tools throughout. We want to understand *how* you worked with them, because that's a real skill we care about. In AI_NOTES.md, briefly cover:

- Which AI tools and models you used, and roughly how you split work between you and the AI.
- 2–3 key decisions you made yourself (architecture, data model, a service choice) and why.
- The single hardest bug or wrong turn the AI led you into — what it got wrong, how you noticed, and how you fixed it. (*This is the part we read most closely. Be specific and honest.*)
- What you'd improve or add with more time.

Optional: include one short prompt or transcript excerpt for the trickiest part if you think it's illuminating. Don't dump full logs.

How we'll evaluate

We weigh, roughly in this order:

1. Does it actually work end-to-end on the live URL — the full flow, not just the happy first step.
 2. Reliability and security of the integration — how it behaves under the unhappy paths described in the quality bar.
 3. Code quality and clarity — structure, readability, sensible choices, a clean repo.
 4. Depth — how far into the stretch goals you got, and how well.
 5. Quality of your AI collaboration — what AI_NOTES.md and your context files reveal about how you think and debug.
-

